Plantilla Formato

DATOS GENERALES ESTUDIANTES

- **NOMBRE: Mateo Gomez Marulanda - Andres Felipe Valencia Arias**
- **DOCUMENTO: 1092458723 - 1028840042**

Escenario 1 — Sistema de Registro de Pacientes

- ¿Usaría la misma estructura para todo?
  - No. Una sola estructura no optimiza todas las operaciones. Se debería usar HashMap para búsquedas rápidas por documento y una Queue o LinkedList para mantener el orden de llegada.
- ¿Cómo priorizar pacientes graves?
  - Usando una PriorityQueue, que ordena a los pacientes según su nivel de gravedad para atender primero los más críticos.
- ¿Qué pasa si llegan 50.000 pacientes al día?
  - La inserción seguiría siendo rápida, pero la búsqueda actual es O(n) y se vuelve lenta. Lo ideal sería usar un HashMap para mantener búsquedas en O(1) incluso con muchos pacientes.

FASE 1 — Comprender el problema

- ¿Qué tipo de datos se manejan?
  - Paciente con atributos como nombre y número de documento.
- ¿Qué operaciones son más frecuentes?
  - Registrar pacientes, buscar pacientes por documento y consultar el orden de llegada.
- ¿Qué operaciones deben ser rápidas sí o sí?
  - La búsqueda por número de documento y el registro de nuevos pacientes.
- ¿El volumen de datos es pequeño, grande o masivo?
  - Grande, ya que un hospital puede recibir decenas de miles de pacientes al día.
- ¿El orden importa?
  - Sí, puesto que es necesario mantener el orden de llegada de los pacientes.

FASE 2 — Selección de la estructura

Operación	Frecuencia	Requisito de eficiencia	Estructura candidata	Justificación
Registrar paciente	**Alta**	**Inserción rápida**	**LinkedHashSet**	**Inserción promedio O(1)**
Mantener orden de llegada	**Alta**	**Mantener orden de inserción**	**LinkedHashSet**	**Mantiene el orden de inserción**
Buscar por documento	**Muy alta**	**Acceso rápido**	**HashMap**	**Acceso promedio O(1)**

Evitar pacientes duplicados	**Alta**	**Validación automática**	**Set / HashSet**	**Acceso promedio O(1)**

FASE 3 — Análisis de complejidad

Operación	Estructura elegida	Complejidad Tiempo	Complejidad Memoria	¿Es aceptable?
Registrar paciente	**LinkedHashSet**	**O(1)**	**O(n)**	**Sí**
Buscar paciente	**HashMap**	**O(1)**	**O(n)**	**Sí**
Mantener orden de llegada	**LinkedHashSet**	**O(1)**	**O(n)**	**Sí**
Evitar duplicados	**Set**	**O(1)**	**O(n)**	**Sí**

Escenario 2 — Plataforma de Ventas Masivas

FASE 1 — Comprender el problema

- ¿Qué tipo de datos se manejan?
  - Productos con atributos como código, nombre, precio y categoría.
- ¿Qué operaciones son más frecuentes?
  - Buscar productos por código, insertar productos nuevos, filtrar por categoría y mostrar productos ordenados por precio.
- ¿Qué operaciones deben ser rápidas sí o sí?
  - La búsqueda por código y la inserción de productos.
- ¿El volumen de datos es pequeño, grande o masivo?
  - Grande o masivo, ya que se insertan miles de productos por hora.
- ¿El orden importa?
  - Sí, aunque solo para mostrar los productos ordenados por precio.

FASE 2 — Selección de la estructura (con justificación)

Operación	Frecuencia	Requisito de eficiencia	Estructura candidata	Justificación
Buscar por código	**Muy Alta**	**Acceso en tiempo constante**	**HashMap**	**Acceso promedio O(1)**

Mostrar orden por precio	**Media**	**Mantener orden automático**	**TreeSet**	**Mantiene el orden / inserción O(log n))**
Insertar productos	**Muy alta**	**Inserción rápida**	**LinkedList**	**Inserción O(1)**
Filtrar por categor	**Alta**	**Acceso rápido por categoría**	**HashMap + List**	**Acceso promedio O(1)**

FASE 3 — Análisis de complejidad (ANTES de programar)

Operación	Estructura elegida	Complejidad Tiempo	Complejidad Memoria	¿Es aceptable ?
Buscar por código	**HashMap**	**O(1)**	**O(n)**	**Sí**
Insertar producto	**LinkedList**	**O(1)**	**O(n)**	**Sí**
Insertar en índice de precio	**TreeSet**	**O(log n)**	**O(n)**	**Sí**
Mostrar ordenados por precio	**TreeSet**	**O(n)**	**O(n)**	**Sí**
Filtrar por categoría	**HashMap + List**	**O(1)**	**O(n)**	**Sí**

Escenario 3

FASE 1 — Comprender el problema (NO programar todavía)

- ¿Qué tipo de datos se manejan?
  - Solicitudes de viaje
- ¿Qué operaciones son más frecuentes?
  - Registrar , Atender, Cancelar y Mostrar
- ¿Qué operaciones deben ser rápidas sí o sí?
  - Registrar
- ¿El volumen de datos es pequeño, grande o masivo?

UNIQUINDÍO
en conexión territorial
www.uniquindio.edu.co

- Pequeño
- ¿El orden importa?
  - Si

FASE 2 — Selección de la estructura (con justificación)

Operación	Frecuencia	Requisito de eficiencia	Estructura candidata	Justificación
Registrar solicitud	Muy Alta	Debe ser Rapida	LinkedList	Inserción promedio O(1)
Atender solicitud	Alta	FIFO	Queue	Acceso promedio O(1)
Mostrar solicitudes	Media	Orden de inserccion	LinkedLIst	Acceso promedio O(1)

FASE 3 — Análisis de complejidad (ANTES de programar)

Operación	Estructura elegida	Complejidad Tiempo	Complejidad Memoria	¿Es aceptable?
Registrar	Queue	O(1)	O(n)	SI
Atender	Queue	O(1)	O(n)	SI
Cancelar	Queue	O(1)	O(n)	SI
Mostrar	LinkedList	O(n)	O(n)	No escala

FASE 4 — Implementación + medición real

1. Implementan la solución
2. Prueban con distintos tamaños de datos (100, 1.000, 10.000, 100.000…)
3. Miden:
   - Tiempo de ejecución
   - Uso de memoria (aproximado)

4. Realizar la conclusión según lo desarrollado en las fases anteriores y los resultados obtenidos.

Escenario 4 — Catálogo de Productos de E-commerce

FASE 1 — Comprender el problema

- ¿Qué tipo de datos se manejan?
  - Producto con atributos como código, nombre y precio.
- ¿Qué operaciones son más frecuentes?
  - Buscar productos por código, insertar productos nuevos y mostrar productos ordenados por precio.
- ¿Qué operaciones deben ser rápidas sí o sí?
  - La búsqueda por código y la inserción de nuevos productos.
- ¿El volumen de datos es pequeño, grande o masivo?
  - Grande, ya que un catálogo de ecommerce puede contener muchos productos.
- ¿El orden importa?
  - Sí, aunque sólo para mostrar los productos ordenados por precio.

FASE 2 — Selección de la estructura

Operación	Frecuencia	Requisito de eficiencia	Estructura candidata	Justificación
Buscar por código	**Muy alta**	**Acceso inmediato**	**HashMap**	**Acceso promedio O(1)**
Mostrar orden por precio	**Media**	**Orden automático**	**TreeSet**	**Mantiene el orden / inserción O(log n)**
Insertar productos	**Alta**	**Inserción eficiente**	**HashMap + TreeSet**	**Inserción O(1) / mantiene orden O(log n)**

FASE 3 — Análisis de complejidad (ANTES de programar)

Operación	Estructura elegida	Complejidad Tiempo	Complejidad Memoria	¿Es aceptable?

Buscar por código	**HashMap**	**O(1)**	**O(n)**	**Sí**
Insertar producto	**HashMap**	**O(1)**	**O(n)**	**Sí**
Insertar para mantener orden	**TreeSet**	**O(log n)**	**O(n)**	**Sí**
Mostrar ordenados por precio	**TreeSet**	**O(n)**	**O(n)**	**Sí**

los programas creados para este laboratorio se encuentran anexados en este archivo via Drive Laboratorio Collections